* Inheritance in Java :-

→ Inheritance is one of the key features of object oriented programming. Inheritance can be defined as the process where one class acquires the properties (Methods and fields) of another class.

→ The idea behind inheritance in Java is that you can create new classes that are built upon existing classes.

→ Inheritance in Java can be best understood in terms of parent and child relationship, also known as super class (parent) and sub class (child) in Java language.

→ Inheritance defines 'is-a' relationship between a super class and its sub class.

→ The main use of inheritance is code reusability.

Syntax of Java inheritance :

```
class subclass-name extends superclass-name
{
  // Methods and fields
}
```

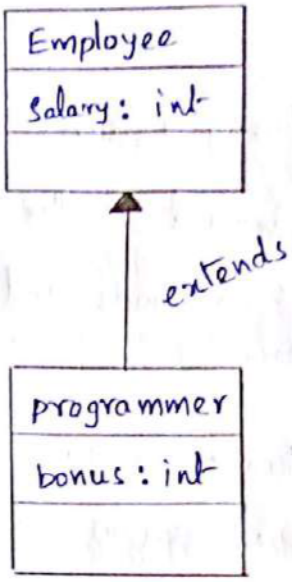
→ In the above syntax, the extends keyword indicates that you are making a new class that derives from an existing class.

→ In simple words, the extends keyword is used to perform inheritance in Java.

→ In the terminology of Java, a class that is inherited is called a super class, the new class is called a subclass.

Scanned by CamScanner

Example :-



→ In the figure, programmer is the sub class and Employee is the super class.

→ Relationship between two classes is programmer IS-A Employee.

programmer.java

```
class Employee
{
  int salary = 4000;
}
Class programmer extends Employee
{
  int bonus = 1000;
  public static void main (String args[])
  {
    programmer p = new programmer ();
    System.out.println ("programmer Salary is :" + p.salary);
    System.out.println (" Bonus of programmer is :" + p.bonus);
  }
}
```

output :-

```
javac programmer.java
java programmer
programmer Salary is : 4000
Bonus of programmer is : 1000
```

Scanned by CamScanner

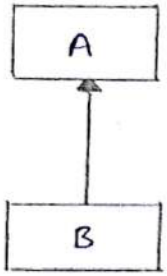
Types of inheritance:

In Java, there can be three types of inheritance those are → Single inheritance
→ Multilevel inheritance
→ Hierarchical inheritance

Note:- Multiple inheritance is not supported in Java.

* Single inheritance:

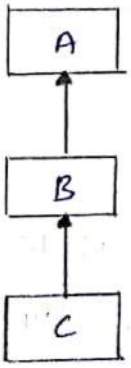
When a class extends another one class only then we call it a single inheritance.



• In the figure, the class B extends the class A. Here A is a parent class and B is a child class.

* Multilevel inheritance:

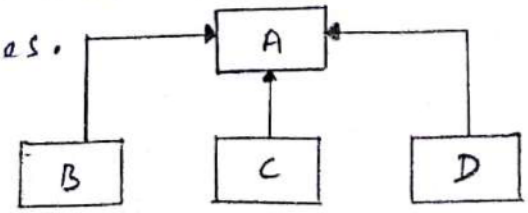
When a class is derived from another class and it acts as the parent class to other class, is known as multilevel inheritance.



• In the figure, the class 'B' inherits properties from class 'A' and again class 'B' acts as a parent to class 'C'.

* Hierarchical inheritance:

In this, one parent class will be inherited by many sub classes.



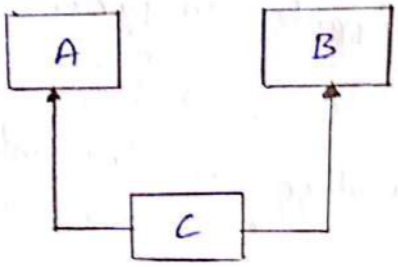
• In the figure, the class 'A' will be inherited by class 'B' class 'C' and class 'D'.

Scanned by CamScanner

We have two more inheritances: Multiple & Hybrid, which are not directly supported by Java.

* Multiple Inheritance:

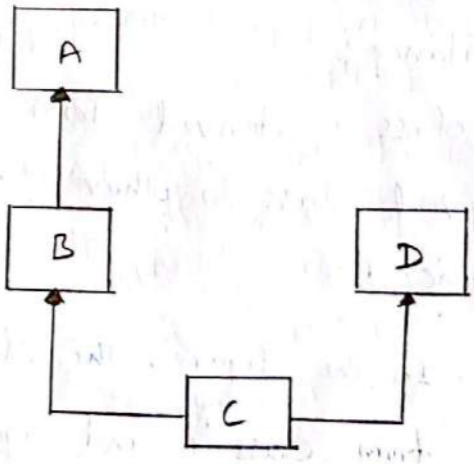
Multiple Inheritance is nothing but one class extending more than one class. It is basically not supported by many Object Oriented programming languages such as Java, Small talk.



Note:- We can achieve multiple inheritance in Java using interfaces.

* Hybrid Inheritance:

Hybrid inheritance is the combination of both Single and Multiple inheritance. It is not directly supported in Java only through interfaces we can achieve this.



Note:-

A class member that has been declared as private will remain private to its class. It is not accessible by any code outside its class, including subclasses.

Scanned by CamScanner

Examples:-

* Single Inheritance Example

```
class A
{
  void dispA()
  {
   System.out.println("disp method of class A");
  }
}
class B extends A
{
 void dispB()
 {
  System.out.println("disp method of class B");
 }
 public static void main(String args[])
 {
   B b = new B();
   b.dispA(); //call dispA() method of class A
   b.dispB(); //call dispB() method of class B
 }
}
```

output:-
```
Javac B.java
java B
disp method of class A
disp method of class B
```

* Multilevel Inheritance Example

```
class A
{
 void dispA()
 {
  System.out.println("disp method of class A");
 }
}
class B extends A
{
```

P.T.O

Scanned by CamScanner

```
  void dispB()
  {
   System.out.println("disp method of class B");
  }
}
class C extends B
{
  void dispC()
  {
   System.out.println("disp method of class C");
  }
  public static void main (String args[])
  {
    C c = new C();
    c.dispA(); // call dispA() of class A
    c.dispB(); // call dispB() of class B
    c.dispC(); // call dispC() of class C
  }
}
```

output:- javac C.Java

java C

disp method of class A

disp method of class B

disp method of class C

* Hierarchical Inheritance Example

```
class A
{
 public void dispA()
 {
  System.out.println("disp method of class A");
 }
}
class B extends A
{
 public void dispB()
 {
  System.out.println("disp method of class B");
 }
}
```

P.T.O

Scanned by CamScanner

```
class C extends A
{
 public void dispC()
 {
  System.out.println("disp method of class C");
 }
}
class D extends A
{
 public void dispD()
 {
  system.out.println(" disp method of class D");
 }
}
class HInheritance
{
 public static void main(String args[])
 {
  B b= new B();
  b.dispB();
  b.dispA();
  C c= new C();
  c.dispC();
  c.dispA();
  D d= new D();
  d.dispD();
  d.dispA();
 }
}
```

output:-

```
Javac HInheritance.java
Java HInheritance
disp method of class B
disp method of class A
disp method of class C
disp method of class A
disp method of class D
disp method of class A
```

Scanned by CamScanner

* Access Modifiers (or) Member access rules in Java :-

The access modifiers in Java specifies accessibility (scope) of a data member, method, constructor or class.

There are 4 types of access modifiers in Java

1. private
2. default
3. protected
4. public

1. private :-

The private access modifier is accessible only within class.

Example :-

```
class A
{
private int data = 40;
private void msg()
{
 System.out.println("Hello Java");
}
}
public class Simple
{
 public static void main (string args[])
 {
   A obj = new A();
   System.out.println(" obj.data);
   obj.msg();
 }
}
```

output:- Javac Simple.Java
compile Time Error
compile Time Error

In the above example, We have created two classes 'A' and 'Simple'. Class 'A' contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

Scanned by CamScanner

2. default:-

If you don't use any modifier, it is treated as default by default. The default modifier is accessible only within package.

Example:-

```
// Save by Adef.java
package pack1;
class Adef
{
 void msg()
 {
  System.out.println("Hello");
 }
}
```

• In This example, we have created two packages pack1 and pack2. We are accessing the Adef class from outside it's package, so it cannot be possible to access from outside the package with default access modifier.

```
// Save by Bdef.java
package pack2;
import pack.*;
class Bdef
{
 public static void main(String args[])
 {
  Adef obj = new Adef(); // Compile Time Error
  obj.msg(); // Compile Time Error
 }
}
```

output: Javac Bdef.java
Compile Time Error.

Example:-

```
class Adef {
  void msg() { System.out.println("Hello"); }
 }
class Bdef {
 public static void main(String args[])
 {
   Adef obj = new Adef();
   obj.msg();
 }
}
```

output:- Javac Bdef.java
Java Bdef
Hello.

Scanned by CamScanner

3. protected :-

The protected access modifier is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

Example :-

// save by ProtectA.java

```
package pack;
public class protectA
{
 protected void msg()
 {
  System.out.println("Hello");
 }
}
```

Note :- In compilation, where -d specifies the destination where to put the generated class file.

* We can use . (dot) to keep the package within the same directory.

// save by ProtectB.java

```
package mypack;
import pack.*;
class protectB extends protectA
{
 public static void main(String args[])
 {
  protectB obj = new protectB();
  obj.msg();
 }
}
```

output :- javac -d . protectA.java
javac -d . protectB.java
java mypack.protectB
Hello.

In the above example, we have created the two packages pack and mypack. In the package pack, the msg method is declared as protected, so it can be accessed from outside the class only through inheritance.

Scanned by CamScanner

4. public :-

The public access modifier is accessible everywhere. It has the widest scope among all other modifiers.

Example :-

```
Package pack;                    // save by X.java
public class X
{
 public void msg()
 {
  System.out.println(" Hello");
 }
}
```

```
package mypack;
import pack.*;          // save by Y.java
class Y
{
public static void main (String args[])
{
  X obj = new X();
  obj.msg();
 }
}
```

output:- javac -d . X.java
javac -d . Y.java
java mypack.Y
Hello

Let's understand the access modifiers by simple table.

Access Modifier	Within class	Within package	outside package by subclass only	outside package
private	Y	N	N	N
Default	Y	Y	N	N
protected	Y	Y	Y	N
public	Y	Y	Y	Y

Note:- Y - Yes, N - No.

Scanned by CamScanner

* Super Keyword :-

The super keyword in Java is a reference variable that is used to refer immediate parent class.

The super keyword is used for

- → Accessing the variables of the parent class
- → calling the methods of the parent (or) super class
- → Invoking the constructors of the parent class.

Example :-

Bike.java

```
class Vehicle
{
  int speed = 50;
  void message()
  {
   System.out.println("Welcome to Vehicle class");
  }
}
class Bike extends Vehicle
{
  int speed = 100;
  void message()
  {
   System.out.println("Welcome to Bike class");
  }
  void display()
  {
   System.out.println("Bike speed is : " + speed); // call variable of local
   System.out.println("Vehical Avg speed is : " + super.speed);
                                                  // variable of parent class
   message(); // invoke local method
   super.message(); //invoke parent class method.
  }
  public static void main(String args[])
  {
    Bike b = new Bike();
    b.display()
  }
}
```

output:- javac Bike.java

```
java Bike
Bike speed is : 100
Vehical Avg speed is : 50
Welcome to Bike class
Welcome to Vehicle class
```

Scanned by CamScanner

→ The super() is used to invoke the parent class constructor

Example :-

Car.java

```
class Vehicle
{
 Vehicle()
 {
  System.out.println("Vehicle is created");
 }
}
class Car extends Vehicle
{
 Car()
 {
  super(); // invoke parent class constructor.
  System.out.println("Car is created");
 }
 public static void main(String args[])
 {
   Car c = new Car();
 }
}
```

output:-

```
javac Car.java
java Car
Vehicle is created
Car is created
```

* Final Keyword :-

The final Keyword in java is used to restrict the user. The final Keyword can be applied on variables, methods and classes.

The Keyword final is used for the following reasons,

→ The final Keyword can be applied on variables to declare constants.

→ The final Keyword can be applied on methods to disallow method overriding

→ The final Keyword can be applied on classes to prevent (or) disallow inheritance.

Scanned by CamScanner

* final variable:

If you declare any variable as final, you cannot change the value of final variable (It will be constant).

Example:- Glamour.java

```
class Glamour
{
final int speedlimit = 100; // final variable
void run()
{
 Speedlimit = 400;
}
public static void main(String args[])
{
 Glamour obj = new Glamour();
 obj.run();
}
}
```

output:- javac Glamour.java

Compile Time Error:
cannot assign a value to final variable

In the above example, we cannot able change the value of variable speedlimit, because it is declared as final.

* final method:

If you declare any method as final, you cannot override that method. It is called as final method.

Example:- Hero.java

```
class Bike
{
final void run() // final method
 {
  System.out.println("running");
 }
}
class Hero extends Bike
{
 void run()
 {
  System.out.println("running safely with 60kmph");
 }
```

Scanned by CamScanner

```
public static void main(String args[])
{
 Hero obj = new Hero();
 obj.run();
}
}
```

output:- javac Hero.java
Compile Time Error:
overridden method is final

* final class:

If you declare any class as final, you cannot extend it. i.e we cannot inherit that class. It is called as final class.

Example:-

Honda.java

```
final class Bike // final class
{
 void run()
 {
  System.out.println("running safely with 60Kmph");
 }
}
class Honda extends Bike
{
public static void main(String args[])
{
 Honda obj = new Honda();
 obj.run();
}
}
```

output:- javac Honda.java
Compile Time Error:
Cannot Inherit from final Bike

* In simple words, The final keyword in Java

→ Stop value change.

→ Stop Method overridding.

→ Stop Inheritance.

Scanned by CamScanner

* Object class :-

In java, there is one special class i.e "Object" class. The object class is the parent (or) super class of all the classes in java by default. In other words, All other classes are sub-classes of object class.

→ It is the topmost class of java.

→ The object class is helpful if you want to refer any object of any class.

The object class defines following methods, which means that they are available in every object.

* object clone():
creates a new object that is the same as the object being cloned.

* boolean equals (object object):
Determines whether one object is equal to another.

* void finalize():
called before an unused object is recycled.

* class getClass():
obtains the class of an object at run-time.

* void wait():
Waits on another thread of execution.

* String toString():
Returns a string that describes the object.

* int hashCode():
Returns the hashcode number for this object.

* void notify():
Resumes execution of thread waiting on the invoking object.

Scanned by CamScanner